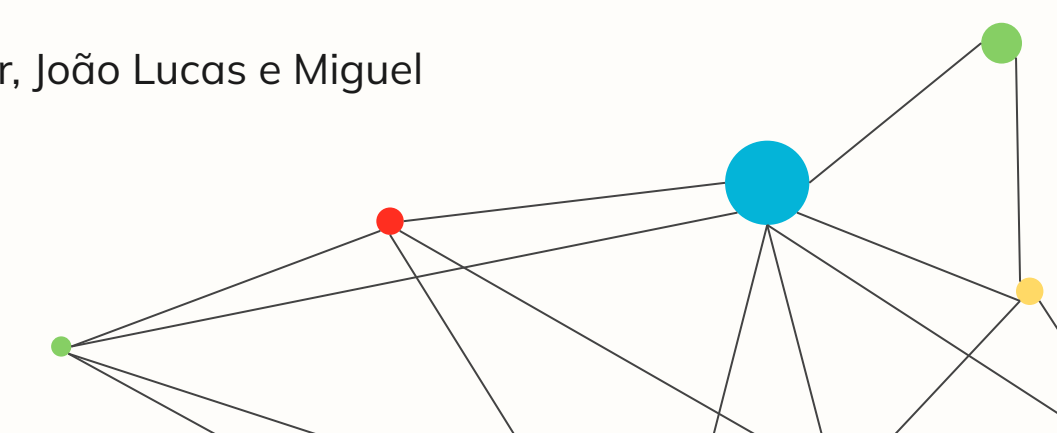
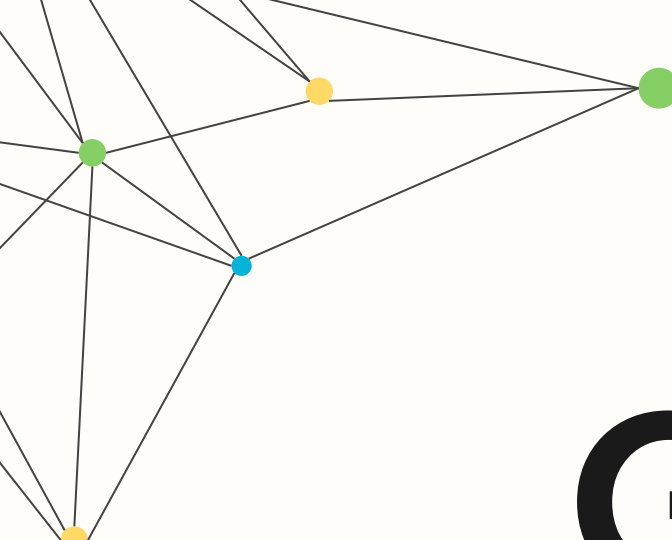
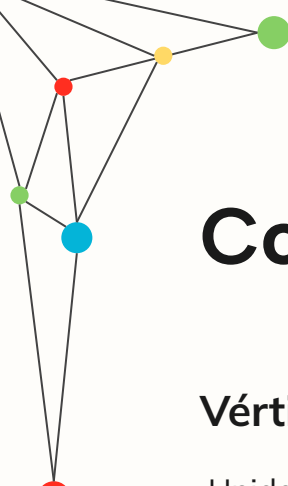




Grafos

Artur Jaeger, João Lucas e Miguel



Conceito

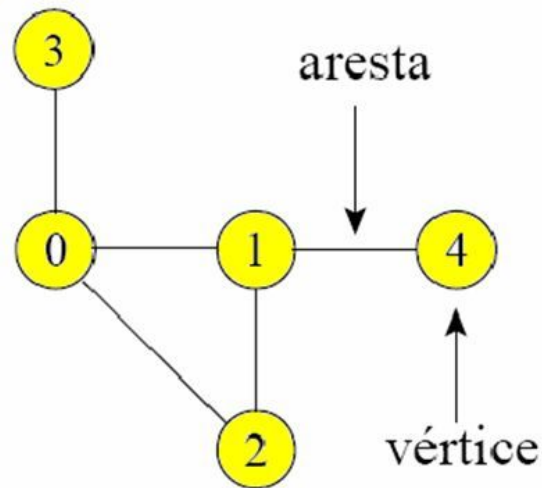
Vértices(nós):

Unidades fundamentais que representam entidades em um grafo (ex: cidades, pessoas, nós de uma árvore).

Arestas(conexões):

Ligação entre vértices:

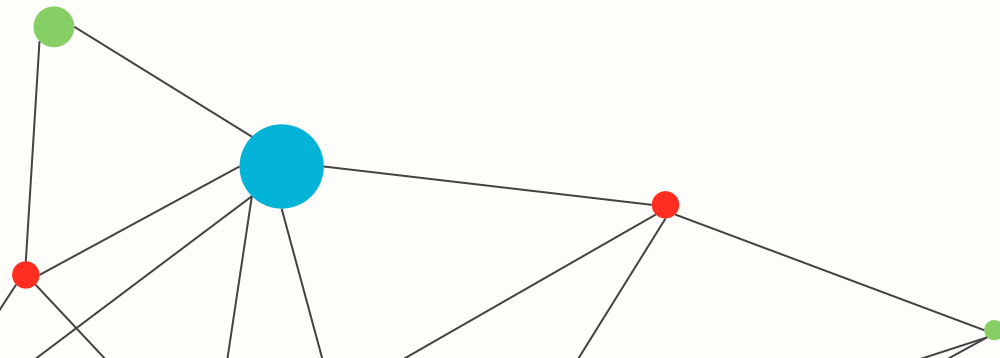
- **Não-direcionadas:** Conexão bidirecional
- **Direcionadas:** Conexão unidirecional
- **Ponderadas:** Possuem um valor associado





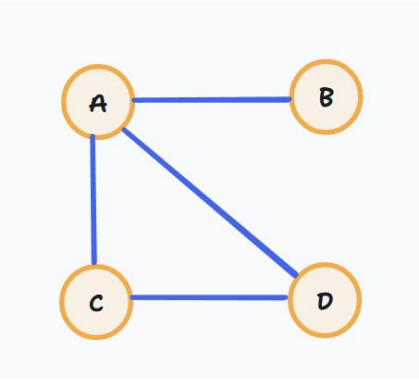
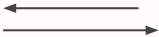
Matrizes de Adjacência

É uma matriz quadrada, booleana com colunas e linhas indexadas pelos vértices. Algoritmo 1 vai representar a existência de uma aresta entre dois vértices.



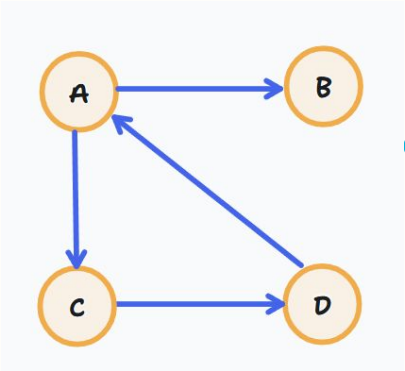
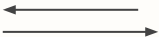
Exemplo 1: grafo não-direcionado

	A	B	C	D
A	0	1	1	1
B	1	0	0	0
C	1	0	0	1
D	1	0	1	0



Exemplo 2: grafo direcionado

	A	B	C	D
A	0	1	1	0
B	0	0	0	0
C	0	0	0	1
D	1	0	0	0





Vantagens e Desvantagens

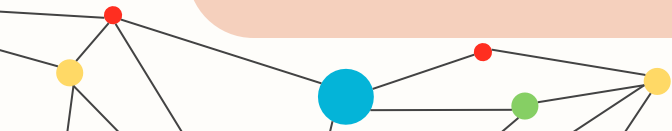
Vantagens

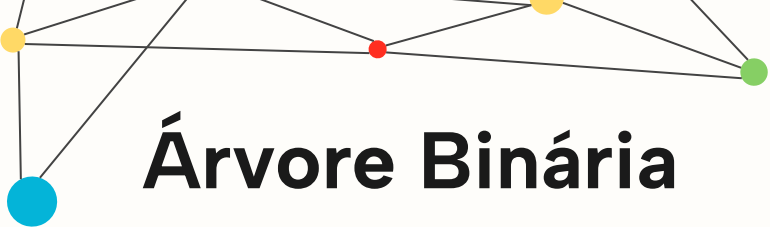
- Consulta rápida de arestas
- Simples de implementar.
- Ideal para grafos densos.

Desvantagens

- Ineficiente para grafos esparsos.
- Inviável para grafos grandes.
- Dificulta interação sobre vizinhos.

Quando usar

- Grafos pequenos e densos.
 - Algoritmos que exigem verificações frequentes de arestas.
 - Implementação de jogos ou simulações com grind.
- 



Árvore Binária

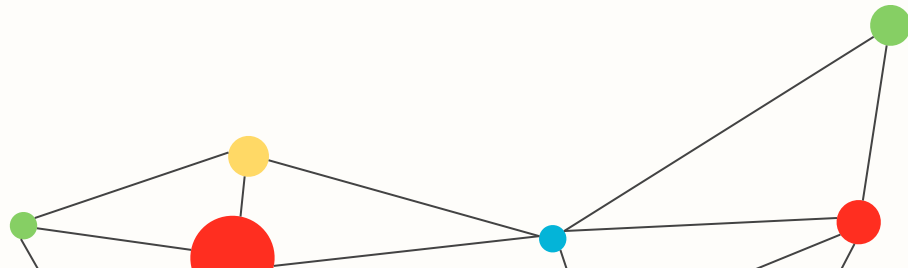
Uma estrutura de dados hierárquica onde cada nó tem no máximo dois filhos(esquerdo e direito).

Característica:

- Raiz: Nó superior da árvore.
 - Folha: Nos sem filho da árvore.
 - Subárvores: Cada nó é uma raiz de uma subárvore.
 - Altura: Comprimento do caminho mais longo até a folha
 - Profundidade: Comprimento do caminho da raiz até um nó específico.
- 

Árvore Binária de Busca(BST)

Nós a esquerda são menores que a raiz e nós a direita são maiores.

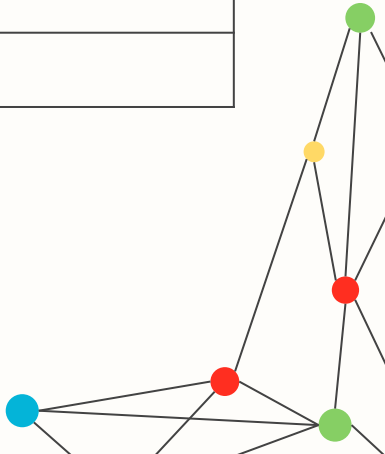




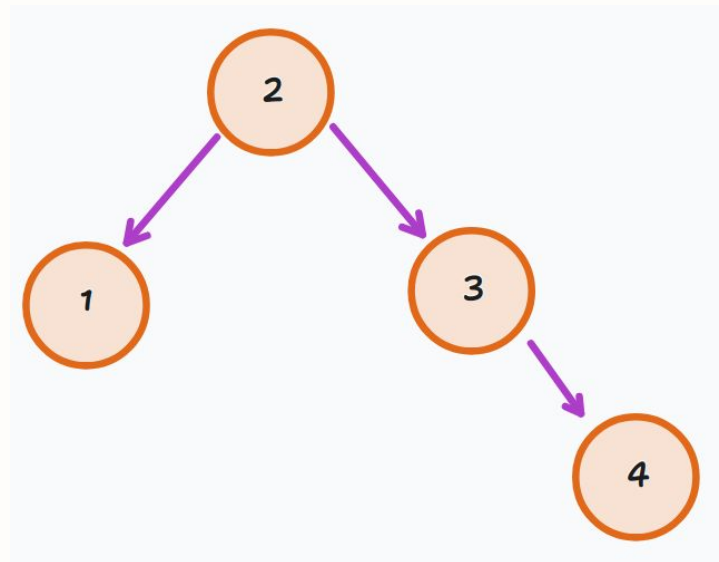
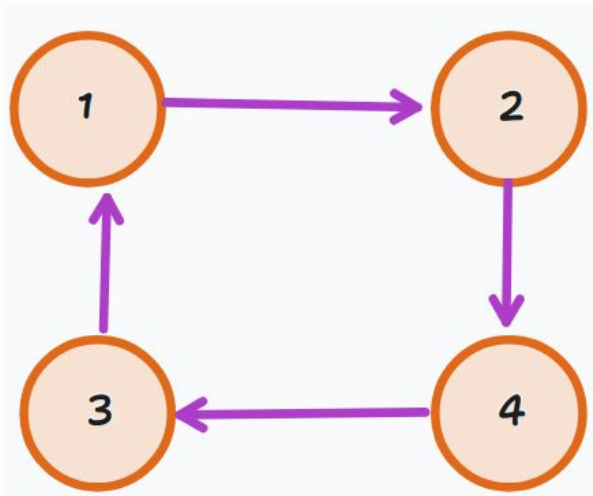
Árvore para representação de grafos

Toda árvore é um grafo, mas nem todo grafo é uma árvore.

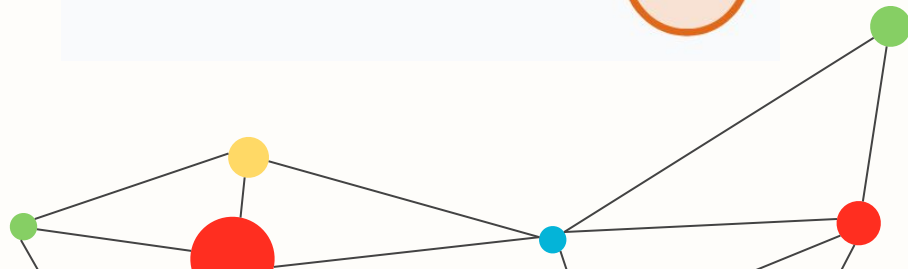
Grafo	Árvore
Pode ter ciclos ou não ter.	Não possui ciclos.
Nem todos os vértices precisam ter uma ligação - desconexo	Todos vertices estao interligados - conexa
Arestas podem ser direcionadas ou nao-direcionadas	Possui uma raiz e uma estrutura de pais e filhos.
Vertices podem ter qualquer grau	Arvore binaria, cada nó possui no máximo 2 filhos.

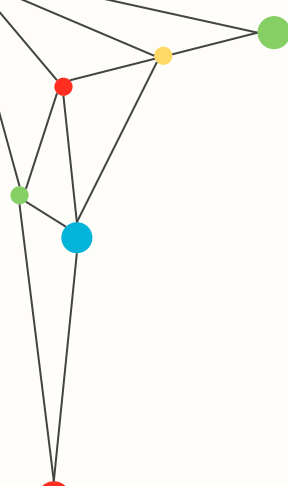


Transformando grafo em uma árvore



Foi escolhido um nó raiz e montado uma hierarquia com menores a esquerda e maiores a direita , para eliminar o ciclo





Tipos de busca em árvores: Profundidade e Largura

Busca em largura (BFS - *Breadth First Search*) e Busca em profundidade (DFS - *Depth First Search*) são algoritmos clássicos para percorrer grafos. Ambos exploram os vértices e arestas, mas de formas distintas: enquanto a BFS explora os níveis de forma ampla, a DFS mergulha o mais fundo possível antes de retroceder.

Busca em largura

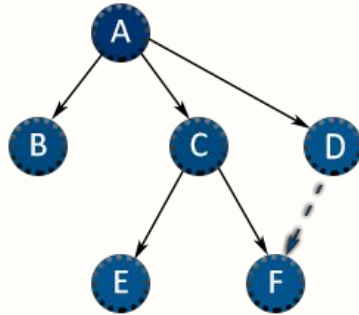
- Usa **fila** para explorar os vértices por nível
- Visita os vizinhos mais próximos antes de ir mais fundo.
- Ideal para encontrar **caminhos mínimos** em grafos não ponderados.
- Útil em **buscas niveladas** e redes sociais.

Busca em profundidade

- Usa **pilha** para ir o mais fundo possível
 - Retrocede ao atingir um vértice sem vizinhos não visitados.
 - Eficiente para **deteccção de ciclos e ordenação topológica**.
 - Comum em **problemas de decisão, jogos, e resolução de labirintos**.
- 

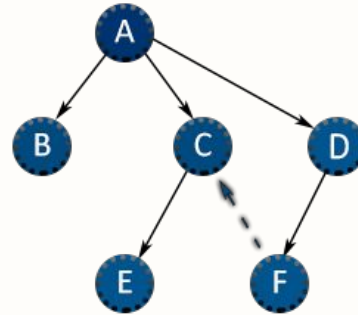
Implementação busca em Profundidade e Largura

BFS



A B C D E F

DFS



A D F C E B



Aplicações: Inteligência Artificial

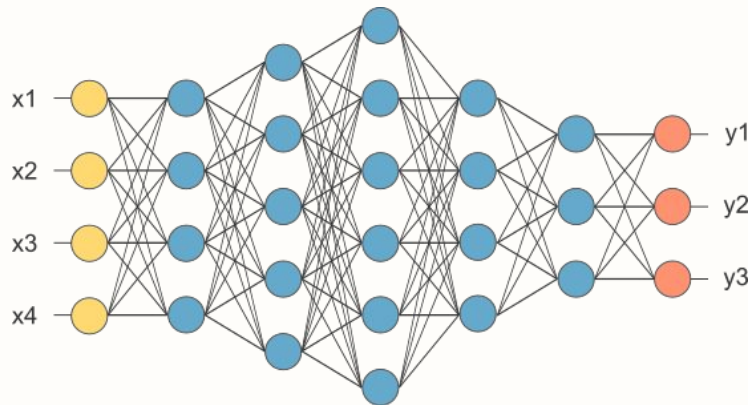
Grafos são estruturas fundamentais para modelar relações e interações entre dados. Eles servem como base para diferentes abordagens em IA, permitindo desde o aprendizado com estruturas complexas até a exploração de caminhos e decisões.

Backtracking (Busca em grafos)

- Resolução de problemas de jogos como Sudoku.
- Busca em profundidade com poda de caminhos inválidos.
- Exploração de estados e transições como vértices e arestas.

Redes Neurais

- Aprendizado em dados não-euclidianos (ex: redes sociais, moléculas).
- Classificação de nós e previsão de links.
- Propagação de informação entre entidades conectadas.





Aplicações: Redes de Computadores

Grafos são fundamentais para modelar e analisar redes, representando dispositivos como nós e conexões como arestas. Essa abstração permite o desenvolvimento de algoritmos eficientes para comunicação, roteamento e gerenciamento de redes.

Topologia e Arquitetura de Redes

- Representação de redes físicas e lógicas (malha, estrela, anel...).
- Análise de conectividade, redundância e tolerância a falhas.
- Planejamento de infraestrutura de rede.

Algoritmos de roteamento

- Roteamento com menor custo (Dijkstra, Bellman-Ford).
- Algoritmos distribuídos em protocolos como OSPF e RIP.
- Cálculo de caminhos alternativos e balanceamento de carga

